

A Parallelized Polynomial Root Finder

Subdivision and the Argument Principle on the GPU

Mohammed Ahmed

July 2026

Abstract

We describe a complex-analytic method for finding *all* roots of a polynomial by recursively subdividing a bounding square and counting the roots in each sub-region with the *argument principle* (winding numbers), then polishing each isolated root with a multiplicity-aware Newton step. The method is deliberately structured as a breadth-first work-list so that it maps directly onto a GPU: each level of the search becomes one kernel launch over a grid of independent cells. We present a CPU reference implementation and three GPU variants that compute the same winding counts with progressively more parallelism—a *naive* per-cell scheme, a *gather* scheme that shares work between neighbouring cells via edge tables, and a *scatter* scheme that fuses the sharing using atomic accumulation. We benchmark all four against NumPy’s companion-matrix eigensolver and a multiprecision reference (`mpmath`), and discuss the limitations—phantom roots, tightly clustered and high-multiplicity roots, and catastrophic cancellation—that the approach shares with, or improves upon relative to, industrial solvers.

1 Introduction and Motivation

Finding the roots of a polynomial

$$P(z) = c_n z^n + c_{n-1} z^{n-1} + \cdots + c_1 z + c_0, \quad c_i \in \mathbb{C},$$

is one of the oldest and most ubiquitous problems in computation. Roots of polynomials appear as the eigenvalues of matrices, the poles and zeros of transfer functions in control theory and signal processing, the intersection points of a ray with an implicit surface in computer graphics, and the solutions of the kinematic equations in robotics. In many of these settings one must solve not one but *millions* of small polynomials—one per pixel, per sample, or per configuration—which is precisely the regime where a graphics processing unit (GPU), with its thousands of parallel threads, can dominate a CPU.

The most widely used method today reformulates the problem as an eigenvalue computation: the roots of P are the eigenvalues of its *companion matrix*, which one finds with a backward-stable QR iteration (this is what `numpy.roots` and MATLAB’s `roots` do). That approach is fast and robust for a single polynomial of moderate degree, but it is inherently sequential and computes *all* n roots whether or not they are wanted.

This report takes a different route, chosen because it *parallelizes naturally*. We enclose all roots in a square, subdivide it, and use a contour integral—the argument principle—to count how many roots lie in each piece. Because the count in each sub-square depends on nothing but that square’s own boundary, every sub-square can be processed by an independent thread. We develop the idea on the CPU as a correctness oracle and then port it to the GPU in three increasingly parallel forms.

We emphasize the scope of this work: the implementation here solves a *single* polynomial at a time. Batching many polynomials into one launch—the setting sketched above, where the GPU most decisively beats the CPU—is a natural extension that we identify as future work rather than a result we report (Section 5).

2 Background

This section collects the classical results the method rests on. A reader comfortable with complex analysis may skip to Section 3.

2.1 The Fundamental Theorem of Algebra

Every polynomial of degree $n \geq 1$ with complex coefficients has exactly n roots in $\mathbb{C}$, counted with multiplicity (FTA). This guarantees that our search is looking for a finite, known number of targets: the counts we compute in each region must sum to n .

2.2 Analytic Functions and Contours

A polynomial is *entire*: it is complex-differentiable everywhere and has no poles. A *contour* is a closed, oriented curve in $\mathbb{C}$; we always use the boundary of an axis-aligned square, traversed counter-clockwise (CCW).

2.3 The Argument Principle

The engine of the whole method is *Cauchy’s argument principle* (link). For a function P analytic inside and on a contour Γ , with no zeros on Γ ,

$$\frac{1}{2\pi i} \oint_{\Gamma} \frac{P'(z)}{P(z)} dz = N, \quad (1)$$

where N is the number of zeros of P enclosed by Γ , counted with multiplicity. Writing $P(z) = |P(z)| e^{i \arg P(z)}$, the integrand is $\frac{d}{dz} \log P = \frac{d}{dz} (\log |P| + i \arg P)$, and the real part integrates to zero around a closed loop. What survives is the change in the *argument*:

$$N = \frac{1}{2\pi} [\text{total change in } \arg P(z) \text{ as } z \text{ traverses } \Gamma \text{ once CCW}] = \frac{1}{2\pi} \Delta_{\Gamma} \arg P. \quad (2)$$

In words: as z walks once around the boundary, the image $P(z)$ winds around the origin exactly N times—the *winding number* of $P \circ \Gamma$. Figure 1 illustrates this.

Discrete winding. We cannot integrate (2) exactly, so we sample S points $z_0, \dots, z_{S-1}$ around the boundary and accumulate the change in argument between consecutive samples:

$$\Delta_{\Gamma} \arg P \approx \sum_{k=0}^{S-1} \text{unwrap}(\arg P(z_{k+1}) - \arg P(z_k)), \quad (3)$$

where `unwrap` folds each increment into $(-\pi, \pi]$. Because `arg` returns a principal value in an interval of width 2π , consecutive raw differences lie in $[-2\pi, 2\pi]$, so a single $\pm 2\pi$ correction always recovers the true (small) increment—*provided the samples are dense enough that the argument never truly changes by more than π in one step*. Dividing the total by 2π and rounding gives the integer count N .

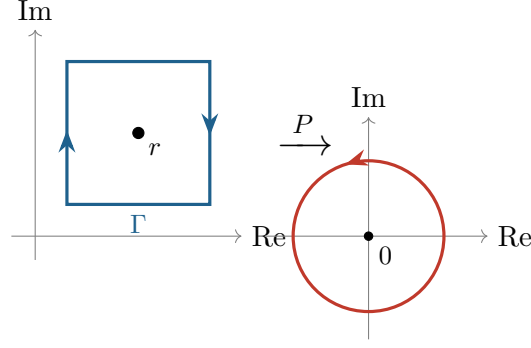


Figure 1: The argument principle. *Left:* the z -plane contour Γ enclosing a root r . *Right:* its image $w = P(z)$, which winds around the origin once for each enclosed root. Here $N = 1$.

2.4 Root-Enclosing Bounds

Before we can subdivide, we need an initial region guaranteed to contain every root. Two classical bounds serve [\(link\)](#):

$$\text{Cauchy: } |z| \leq 1 + \max_{0 \leq k < n} \left| \frac{c_k}{c_n} \right|, \quad (4)$$

$$\text{Fujiwara: } |z| \leq 2 \max_{1 \leq k \leq n} \left| \frac{c_{n-k}}{c_n} \right|^{1/k}, \text{ with the } k = n \text{ term halved.} \quad (5)$$

Both are valid upper bounds, so their minimum $R = \min(\text{Cauchy}, \text{Fujiwara})$ is too. Every root satisfies $|z| \leq R$, so the square $[-R, R] \times [-R, R]$ encloses all of them. (These bounds are themselves Rouché-type arguments: on a large enough circle the leading term $c_n z^n$ dominates the rest.)

2.5 Newton's Method and Basins of Attraction

Once a region has been narrowed to a single root, we refine it with *Newton's method* [\(link\)](#):

$$z_{k+1} = z_k - \frac{P(z_k)}{P'(z_k)}. \quad (6)$$

Newton converges quadratically near a simple root. However, its *basins of attraction*—the sets of starting points converging to each root—are *fractal* ([Newton fractal](#)): a seed geometrically close to one root can converge to a distant one. This subtlety turns out to matter (Section 5). For a root of multiplicity m , the modified iteration $z_{k+1} = z_k - m P/P'$ restores quadratic convergence.

3 Method

The overall algorithm is a subdivision search:

1. Compute the bound R and start with the square $\Gamma_0 = [-R, R] \times [-R, R]$ (nudged by a tiny offset so no root lands exactly on a subdivision line, where the winding integral is undefined).
2. Maintain a work-list of squares. For each square, compute its winding count N via [\(3\)](#).
3. **Triage** each square by its count:

- $N = 0$: no roots—*discard* the square (a “dead” square).
 - $N = 1$ and the square is small enough: the root is *isolated*—send it to Newton polishing.
 - $N \geq 2$ (or $N = 1$ but still large): *subdivide* the square into four and re-count each child.
4. Repeat until every surviving square is isolated, then polish.

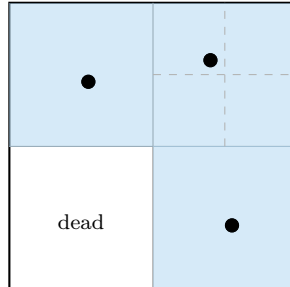
The invariant that makes the search sound is *conservation*: by the argument principle the counts of a square’s four children must sum to the parent’s count, so no root is created or destroyed as we descend. Crucially, the work at one level is a set of *independent* winding computations—which is exactly what a GPU wants.

3.1 CPU Method

The CPU reference uses *binary* 2×2 subdivision driven by a breadth-first work-list; each iteration of the outer **while** loop processes one whole level. Two robustness features matter:

- **Adaptive sampling.** The number of boundary samples starts at $\sim 4n$ per side and *doubles* whenever the computed winding is suspiciously far from an integer (a sign of undersampling near a grazing root).
- **Noise-floor trust.** When a square shrinks onto a multiple root, $|P|$ collapses toward the rounding-error floor $\varepsilon_{\text{mach}}$ · (largest Horner partial); there $\arg P$ is meaningless. Detecting this stops the search from “burrowing into garbage.”

Figure 2 shows the subdivision as a quadtree over the complex plane.



squares with a nonzero count (shaded) are subdivided;
empty squares (“dead”) are pruned

Figure 2: Subdivision as a quadtree. Each square’s winding count decides whether it is pruned, subdivided, or handed to Newton.

3.2 GPU Methods

On the GPU, one level of the search is a grid of independent winding computations. The three variants below all produce the *same* per-cell counts; they differ only in how they distribute the arithmetic. The dominant cost in every case is evaluating P (by Horner’s method, $O(n)$ work) at each of the $\sim S$ boundary samples of each cell.

3.2.1 Naive Method

The simplest scheme lays an $m \times m$ grid over the region and assigns **one thread to one cell**. Each thread walks all four sides of its cell, accumulates the argument change (3), and writes the integer count. Cells with count 0 are “dead” and dropped; cells with count > 0 are “live” and passed to the next level. Figure 3 shows a grid with its live and dead cells.

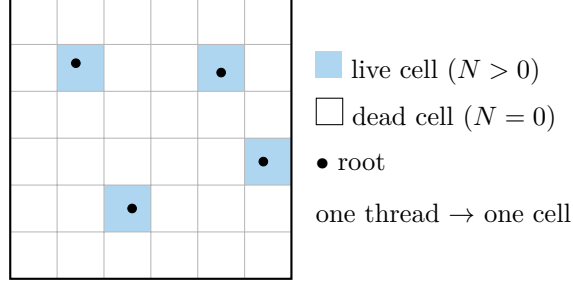


Figure 3: Naive GPU winding. Each thread computes the winding count of one grid cell independently. Shaded cells are “live” (contain roots) and survive to the next level; the rest are pruned.

The naive scheme is simple but wasteful: every *interior* edge of the grid is shared by two adjacent cells, and each cell recomputes its own four edges. So each interior edge—and the $O(n)$ Horner evaluations along it—is computed *twice*.

3.2.2 Gather (H/V) Method

The gather method removes that redundancy. Observe that an $m \times m$ grid has about $4m^2$ cell-sides but only about $2m^2$ *unique* edges (each interior edge is shared). So we compute each unique edge *once* and reuse it:

1. **Edge pass.** One thread per unique edge. Each thread integrates the argument change along its edge (in a fixed canonical direction: horizontal edges left-to-right, vertical edges bottom-to-top) and stores the result—a real number, the edge’s *phase change*—in a table: $H[r][c]$ for horizontal edges, $V[r][c]$ for vertical edges.
2. **Assembly pass.** One thread per cell. Each cell gathers its four edges from the tables and combines them with signs dictated by a CCW traversal:

$$N(r, c) = \text{round} \left(\frac{H[r][c] + V[r][c+1] - H[r+1][c] - V[r][c]}{2\pi} \right). \quad (7)$$

Because each edge is now evaluated once instead of twice, the gather method does about **half** the Horner work of the naive method—the $2m^2$ vs. $4m^2$ ratio. It uses no atomics and is fully deterministic. Figure 4 shows the shared edges and the four edges of one cell.

3.2.3 Scatter (Atomic) Method

The scatter method achieves the same reuse but *fuses* the two passes. Instead of each cell *gathering* its edges, each **edge thread** computes its phase once and *scatters* it into the two cells that share it, via an atomic add. Using the CCW sign convention, a vertical edge adds $+$ to the cell on its left and $-$ to the cell on its right; a horizontal edge adds $+$ to the cell above it and $-$ to the cell below (Figure 5). A per-cell accumulator M receives these contributions; a final pass divides by 2π and rounds.

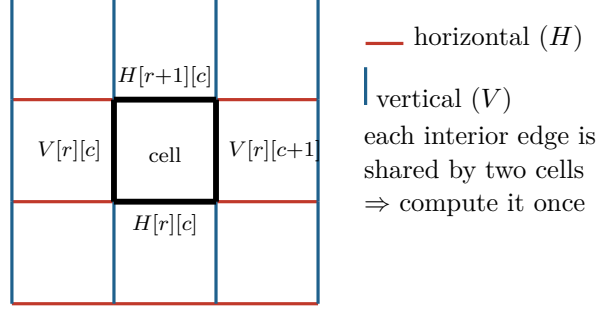
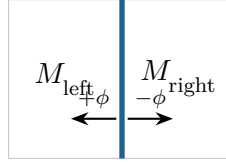


Figure 4: Gather method. Every unique edge (red H , blue V) is integrated once; a cell’s count is assembled from its four cached edges via (7).



one edge computes its phase ϕ once, then `atomicAdds`
 $+\phi$ and $-\phi$ into its two neighbouring cells

Figure 5: Scatter method. Each edge scatters its phase into the two cells it borders with opposite signs; the assembly is fused into the scatter.

Scatter vs. gather trade-off. Scatter needs *less memory*: it keeps only the per-cell accumulator M (m^2 numbers) rather than the two edge tables H and V ($\approx 2m^2$ numbers), and it avoids a separate assembly kernel. The cost is *determinism*: atomic additions arrive in a nondeterministic order, and floating-point addition is *not associative* (the rounding of $a+b+c$ depends on the order), so the accumulated M —and, at a rounding boundary, the final count—can wobble in the last bits from run to run. For well-conditioned inputs this is invisible; near a multiple root, where the counts are already unreliable, the two schemes can even disagree.

4 Results

We benchmarked the CPU solver and the three GPU variants against `numpy.roots` (a LAPACK companion-matrix eigensolver—the practical baseline) and `mpmath.polyroots` (arbitrary precision—an accuracy oracle). All methods share one input/output contract and run in the same environment. Table 1 shows representative cases; “LOST” means a true root was dropped, and errors are the worst distance to a true root.

Three findings stand out.

- **The GPU is far faster than the CPU** on well-conditioned problems—roughly 10–75 $\times$ at these degrees (e.g. 43 ms vs. 3260 ms at degree 50)—validating the port against its own baseline.
- **NumPy remains the overall winner** at these degrees: it is the fastest *and* robust. The GPU’s fixed launch and data-transfer overhead means it can only overtake NumPy at much higher degree, or—its natural regime—when solving many polynomials in a single *batch*.

Polynomial	Method	Roots found	Max error	Time (ms)
Roots of unity, $z^{50} - 1$	CPU	50 / 50	9×10^{-16}	3260
	GPU (gather)	50 / 50	9×10^{-16}	43
	NumPy	50 / 50	2×10^{-15}	2.0
	mpmath	50 / 50	9×10^{-16}	1650
Wilkinson, $\prod_{k=1}^{20} (z - k)$	CPU	LOST (3)	—	146
	GPU (gather)	20 / 20	9×10^{-3}	12
	NumPy	20 / 20	2×10^{-2}	0.6
	mpmath	20 / 20	6×10^{-4}	435
Tight cluster (gap 10^{-6})	CPU	4 / 4	4×10^{-8}	3
	GPU (gather)	4 / 4	9×10^{-12}	6
	NumPy	4 / 4	1×10^{-10}	0.3

Table 1: Representative benchmark results. The three GPU variants return identical roots and differ only in speed (gather shown).

- **The subdivision method is often *more robust* than the CPU’s own weaknesses would suggest**, and on several hard cases (Wilkinson, tight clusters, wide dynamic range) the finer GPU grid recovers roots the CPU drops.

Among the three GPU variants the counts are identical, so accuracy is the same; the differences are purely in speed, where the gather scheme’s elimination of redundant edge work makes it the fastest of the three at scale.

5 Limitations

The method has real limitations, most of which trace back to the finite precision of floating-point arithmetic.

Phantom roots. Near a high-multiplicity root, $|P| \sim r^m$ collapses toward the rounding-noise floor before the enclosing square becomes small. There $\arg P$ is dominated by round-off, so the discrete winding integral (3) returns a *wrong* integer. In the worst case the search subdivides noise into spurious “phantom” boxes—for example $(z - 1)^{24}$ can report a total multiplicity of 42 instead of 24—or loops until a work-list cap halts it. A cap guarantees termination, but a genuinely robust fix requires raising the noise floor (compensated Horner) or replacing the integer winding with a *certified* count via Rouché/Pellet tests, which fail *safe* (inconclusive $\rightarrow$ subdivide) rather than fail *wrong*.

Clusters and high multiplicity. A root of multiplicity m is intrinsically ill-conditioned: perturbing the coefficients by ε moves the root by $\sim \varepsilon^{1/m}$. In double precision this caps the achievable accuracy at $\sim \varepsilon_{\text{mach}}^{1/m}$ —about 10^{-8} for a double root, 10^{-5} for a triple. This limit is *universal*: NumPy hits the same ceiling. The difference is the failure *mode*—the companion-matrix method degrades gracefully (it always returns n points, just imprecise ones), whereas subdivision can *miscount* at the noise floor.

Catastrophic cancellation. When the roots span a wide dynamic range (say 10^{-3} to 10^3), Horner evaluation near the small roots suffers catastrophic cancellation: the partial sums are large

while $|P|$ is tiny, so the relative error explodes and $\arg P$ becomes unreliable. Empirically the method tolerates only ~ 2 – 3 orders of magnitude of spread. Wilkinson’s polynomial $\prod_{k=1}^{20} (z - k)$ ([link](#)) is the canonical example of this coefficient ill-conditioning, and it defeats every double-precision method to some degree.

A Newton-basin pitfall. Isolating a single-root cell does not guarantee Newton converges to *that* root: because Newton basins are fractal (Section 2.5), a seed at the cell’s centre can escape across a basin boundary to a neighbouring root—catastrophically so for symmetric, densely packed roots like roots of unity. The fix is to subdivide until the cell is small enough (a threshold that must shrink like $1/n$ with the degree) that the centre is safely inside the correct basin.

GPU utilization. A single small polynomial simply does not contain enough parallel work to fill a GPU: a degree-50 solve has only ~ 50 active cells per level, far short of the tens of thousands of threads a GPU wants resident. The method’s true GPU regime is therefore *batch throughput* (many polynomials at once) and very high degree—not single-polynomial latency, which is the GPU’s least favourable framing. This work does *not* yet implement batching: every solve here processes one polynomial, and turning the per-polynomial pipeline into a single batched launch is the most promising line of future work.

6 References

- Cauchy’s argument principle — https://en.wikipedia.org/wiki/Argument_principle
- Fundamental theorem of algebra — https://en.wikipedia.org/wiki/Fundamental_theorem_of_algebra
- Bounds on polynomial roots (Cauchy, Fujiwara) — https://en.wikipedia.org/wiki/Geometrical_properties_of_polynomial_roots
- Newton’s method and Newton fractals — https://en.wikipedia.org/wiki/Newton_fractal
- Wilkinson’s polynomial (ill-conditioning) — https://en.wikipedia.org/wiki/Wilkinson%27s_polynomial
- Rouché’s and Pellet’s theorems (certified root counting) — https://en.wikipedia.org/wiki/Rouch%C3%A9%27s_theorem
- D. A. Bini and G. Fiorentino, *Design, analysis, and implementation of a multiprecision polynomial rootfinder* (MPSolve), Numer. Algorithms, 2000.
- Companion matrix / eigenvalue method (as used by `numpy.roots`) — https://en.wikipedia.org/wiki/Companion_matrix